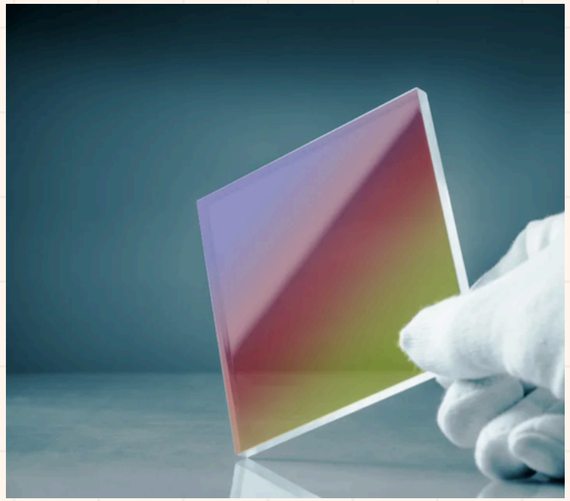
A RAG-BASED GENERATIVE AI FRAMEWORK FOR THE AUTOMATED EXTRACTION AND OPTIMIZATION OF SPUTTERING PARAMETERS IN FUNCTIONAL THIN FILMS

UNDERGRADUATE PROJECT (2025-26 EVEN SEMESTER)



Vatsal Kumar (IIT Kanpur)
Under the guidance of Prof. Shikhar Misra,
Dept. of Material Science and Engineering,
IIT Kanpur

THE PROBLEM STATEMENT

- **Unstructured Data:** Decades of critical sputtering parameters (Power, Pressure, Temperature) are locked inside unstructured PDF research papers, requiring hundreds of hours of manual literature review.
- **Data Fragmentation (Semantic Inconsistency):** Researchers use varying chemical aliases and formatting (e.g., "ZnO" vs. "Zinc Oxide" vs. "glass" vs. "Glass"), making aggregate statistical analysis across papers nearly impossible.
- **The AI Shortcoming (Hallucination):** While standard Large Language Models (LLMs) can read text, they struggle with deterministic math and often hallucinate when asked to calculate strict engineering baselines (Mean, Median, Mode).
- **The Missing Link:** There is no unified platform that bridges the gap between Macro-level statistical consensus (broad industry baselines) and Micro-level precision (interactive, hallucination-free Q&A for specific papers).

PROPOSED ARCHITECTURE

DUAL-STORAGE HYBRID RAG SYSTEM

AN INTEGRATED PIPELINE SEPARATING SEMANTIC REASONING FROM DETERMINISTIC MATHEMATICS.

Semantic Retrieval (ChromaDB)

- Function: Acts as the contextual search engine.
- Process: Maps user queries (e.g., "ZnO parameters") into a high-dimensional vector space to instantly retrieve the top 50 most contextually relevant unstructured papers.

Dynamic Entity Normalization (Gemini LLM)

- Function: Acts as the semantic filter to solve data fragmentation.
- Process: The LLM reads the retrieved papers and dynamically groups all chemical aliases, spelling variations, and typos (e.g., ZnO, Zinc Oxide, ZnO) into a single, clean, verified dataset.

Deterministic Analytics (Pandas Engine)

- Function: Acts as the mathematical calculator to eliminate AI hallucination.
- Process: Processes the AI-cleaned dataset to calculate strict, mathematically guaranteed consensus metrics (Mean, Median, Mode, Range) across the entire database

DATA EXTRACTION PIPELINE

Phase 1: Raw Data Ingestion

- Sourced thousands of unstructured, visually complex research papers (PDFs) detailing thin film deposition and sputtering

Phase 2: Structural Parsing (GROBID)

- Utilized the GROBID machine learning library to convert rigid PDFs into structured XML/JSON formats, accurately isolating critical text blocks (Abstracts, Experimental Methods, Results)

Phase 3: AI-Driven Parameter Extraction

- Engineered a fault-tolerant Python script (with automated API key rotation) using the Gemini LLM.
- The AI acts as a targeted reader, scanning the parsed text to extract 10 highly specific sputtering parameters (Target Power, Working Pressure, Substrate, etc.) into a strict, validated JSON schema

Phase 4: Data Normalization & Post-Processing (The "Transform" Step)

- Unit Standardization: Enforced uniform metrics (e.g., Watts for Power, Celsius for Temp, Pascals for Pressure).
- Data Coercion: Converted text artifacts (e.g., "Not specified") into NaN (Null) values to allow for safe mathematical operations.
- String Cleaning: Standardized capitalization and stripped whitespace to prevent data fragmentation (e.g., merging "glass" and "Glass")

Phase 5: Dual-Storage Loading

- The cleaned master dataset is compiled into a CSV for Pandas (Statistical Engine) and embedded into ChromaDB (Semantic Vector Space

ENGINEERING THE MICRO-RAG CHATBOT

Context Injection (The "R" in RAG)

- Instead of letting the AI guess, the system queries ChromaDB to find the top 10 most relevant paper chunks and injects them directly into the LLM’s short-term memory before it answers.

Automated Source Traceability

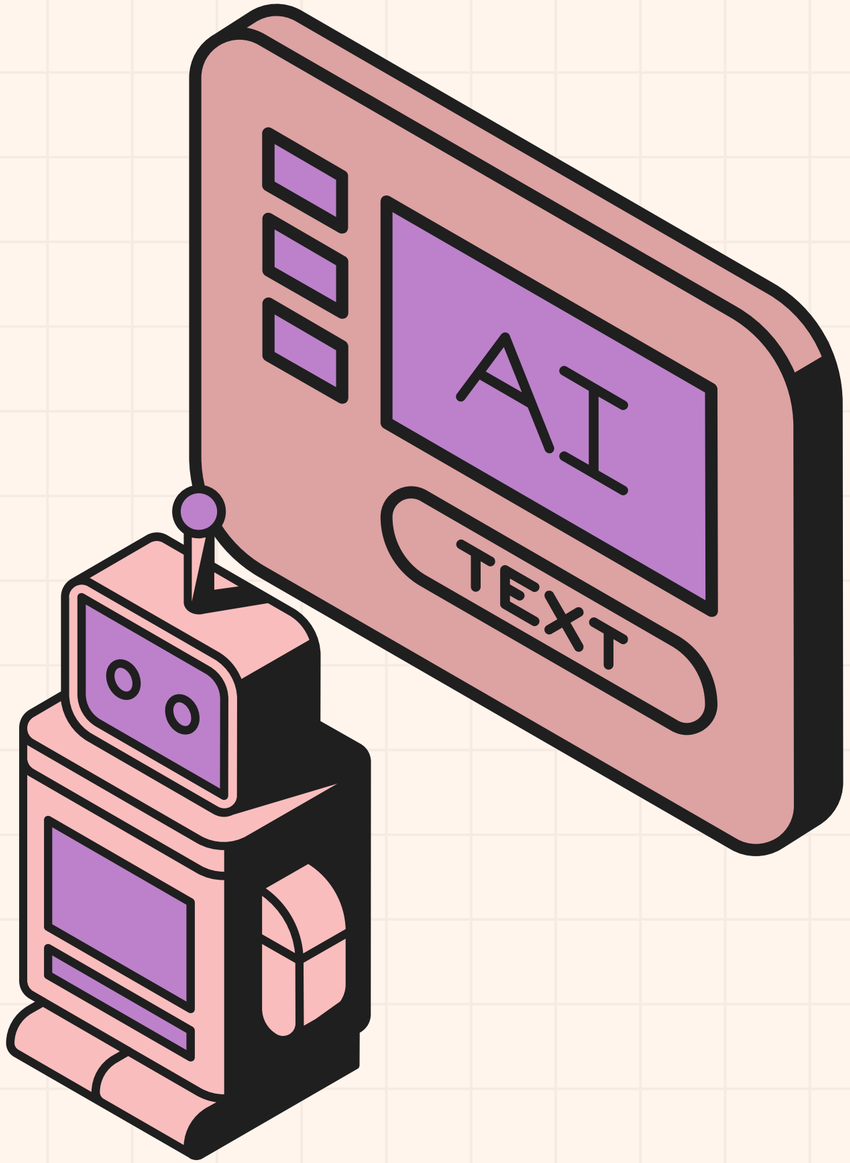
- Academic Integrity: The script intercepts the vector metadata and automatically appends the exact Paper IDs and Material Names to the bottom of every generated response, ensuring users can independently verify the AI's claims.

Severe Prompt Guardrails

- Engineered strict system prompts to constrain the Gemini model's behavior:
  - **Absolute Grounding**: The AI is forbidden from using outside internet knowledge. If the answer is not in the injected text, it must explicitly state: "I cannot find the answer in the current database."
  - **Data Omission Rule**: The model is programmed to automatically filter out null values (e.g., "Not specified", "NaN") to keep outputs strictly quantitative.

THE MACRO VIEW

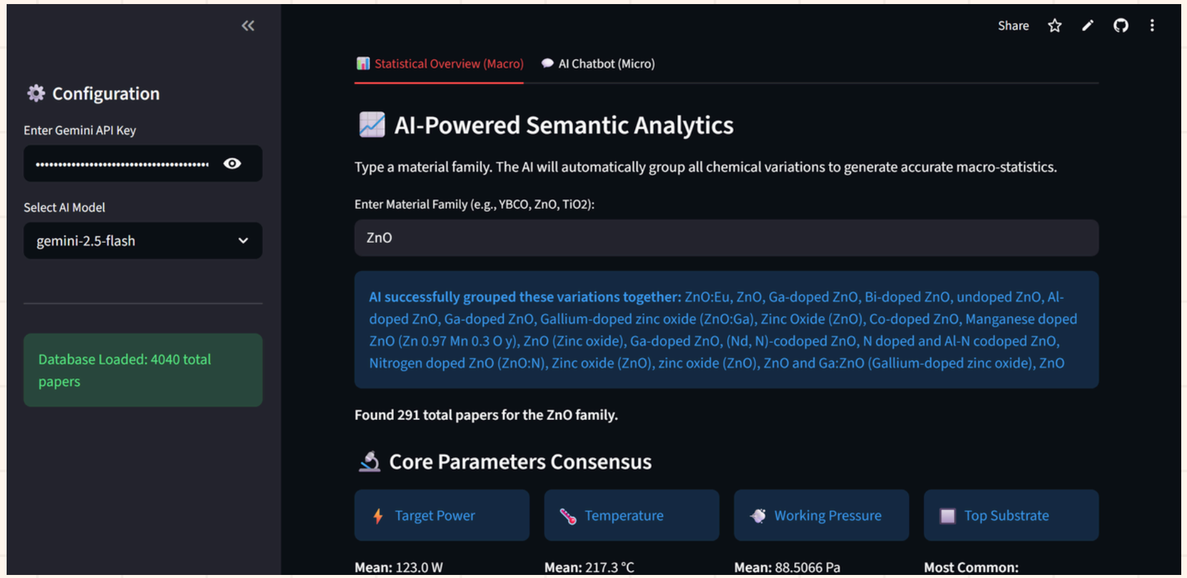
- **Dynamic Semantic Grouping:** The user inputs a base material (e.g., "ZnO"), and the AI silently clusters all chemical aliases and variations in the background.
- **Instant Consensus Metrics:** Pandas calculates the mathematically flawless working range, mean, median, and mode for Target Power, Temperature, and Working Pressure.
- **Visual Distributions:** Real-time histograms allow researchers to visually identify the industry-standard substrate preferences and power distributions at a glance.



DASHBOARD DEPLOYMENT & TECH STACK

The UI Framework (Streamlit)

- Deployment: Deployed as a dynamic, pure-Python web application using the Streamlit framework.
- Why Streamlit? Allows for rapid frontend prototyping specifically designed for Machine Learning and Data Science data visualization, without needing complex HTML/CSS.



Backend Integration & APIs

- The Brain: Directly integrates the Gemini API for real-time inference and entity resolution.
- The Memory: Connects locally to the persistent ChromaDB vector database to ensure zero-latency semantic retrieval.
- The Engine: Uses Pandas for instant, backend mathematical calculations and Plotly for rendering interactive UI graphs.

Execution Flow (How it works)

- Input: User enters a material name or a specific question via the Streamlit interface.
- Routing: The application logic automatically routes the query to either the Pandas Engine (for Tab 1 math) or the RAG Pipeline (for Tab 2 chat).
- Render: The backend calculates the response and pushes it back to the Streamlit UI, updating the charts and text in real-time.

Dashboard Link- https://sputtering-rag-ai.streamlit.app/
Github Repository - https://github.com/vatsal3512/sputtering-rag-ai

CONCLUSION

- End-to-End Automation: Successfully engineered a fault-tolerant ETL pipeline (GROBID + Gemini) to extract unstructured PDF data into a structured database, eliminating hundreds of hours of manual labor.
- Overcame AI Limitations: Solved the "hallucination" problem by building a Hybrid RAG architecture that strictly separates semantic grouping (LLM) from deterministic mathematics (Pandas).
- Deployed a Practical Solution: Delivered a fully interactive, production-ready Streamlit dashboard bridging Macro-level statistics and Micro-level paper retrieval.

FUTURE SCOPE

- Database Expansion: Scale the automated extraction pipeline to process 10,000+ papers, incorporating other deposition methods like CVD (Chemical Vapor Deposition) and ALD.
- Open-Source Contribution: Publish the cleaned, structured CSV and Vector Database for the wider academic community.

THANK YOU